Day 112: Support Vector Machines From Scratch — Separable and Non-Separable Data

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 1 | Complete study guide

Watch: Support Vector Machines Part 1 (of 3): Main Ideas!!! (StatQuest) | Support Vector Machines in Python from Start to Finish (StatQuest)

Script: `day112_support_vector_machines.py` · every number in this guide is copied by program from `day112_run_output.txt`, the real output of that script.

A support vector machine written from scratch, with its dual problem solved by SMO. It matches scikit-learn's SVC on every test tumour (1.0000 agreement, cosine 0.999998 between the two weight vectors), and the duality gap is 0.013% of the objective. On separable iris flowers it finds the widest possible gap (1.3038 cm) where the perceptron lands anywhere from 0.0109 to 1.2132 cm. Its defining property holds: only the support vectors matter. Against logistic regression on breast cancer it is no better (-0.0023). Unscaled, its solver failed to finish on 19 of 20 splits.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

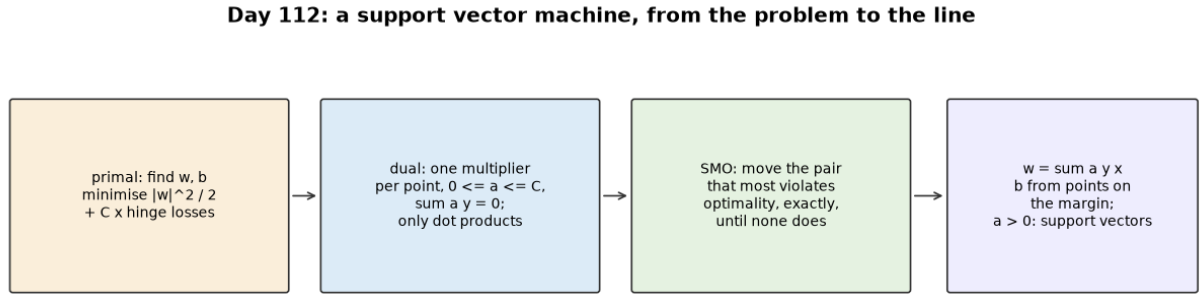
A **support vector machine** (SVM) separates two classes with the straight boundary that leaves the widest **margin**: the gap between the boundary and the nearest points of either class. Those nearest points are the **support vectors**; no other point affects the answer.

When the classes overlap, the **soft margin** lets points sit inside the gap or on the wrong side, at a cost **C** per unit of violation: large C insists on fitting the training points, small C prefers a wide margin.

REAL-WORLD SCENARIO

Painting a road between two fields: you want the widest road that touches neither field. Only the fence posts nearest the road decide where it goes; moving a post far from the road changes nothing.

SYSTEM DIAGRAM



From the problem to the line: primal, dual, SMO, and the weights recovered from the multipliers.

Today's part for the week's build

`day112_exercise.py` is the solver alone: it sees only a matrix of dot products, so Day 113 can hand it other kernels. Week 17's build on Day 118 will compare the week's models with cross-validation. Its real output:

ACTUAL OUTPUT

```
multipliers [0.0, 0.8889, 0.8889, 0.0] after 4 steps; sum a y = 0.0e+00
w = 1.3333, b = -1.6667: the boundary is at x = 1.2500, halfway between 0.5 and 2 (by hand: 1.25,
    w = 2 / 1.5 = 1.3333)
the point at x = 10 is far from the boundary, so its multiplier is exactly 0.0; the one at x = 0
    too: 0.0
decision values for x = 1 and x = 1.5: [-0.3333, 0.3333]
```

Data: iris and breast cancer, both bundled with scikit-learn. The SVM is written in NumPy; scikit-learn's SVC is run only as an outside check.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: two points

DEFINITION — start here, no prior knowledge assumed

With one point of each class, the best boundary is the perpendicular bisector of the segment between them, and the margin is the distance between the points. Both points are support vectors, sitting exactly on the edges of the gap: y(w·x + b) = 1.

REAL-WORLD SCENARIO

Two houses on opposite sides of a street: the fairest boundary is the line exactly halfway between them.

MATHEMATICS

$$\text{minimise } \tfrac{1}{2}|w|^2 \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1 \text{ for every point}$$

$$\text{margin} = 2 / |w|$$

ACTUAL OUTPUT

```
points (1, 1) labelled -1 and (3, 3) labelled +1: the best line is halfway, at right angles to the
    segment between them
w = [0.5, 0.5], b = -2.0000, multipliers [0.25, 0.25]
by hand: w = (0.5, 0.5), b = -2, margin 2 / |w| = 2.8284 = the distance between the points; from
    the solver 2.8284
both points are support vectors: y (w.x + b) = [1.0, 1.0]
Actual output from this lesson's run.
```

What it means

The solver gives w = (0.5, 0.5) and b = –2, exactly as by hand, and a margin of 2.8284, the distance between (1, 1) and (3, 3). Both multipliers are 0.25: equal shares, since both points hold the boundary in place.

WHY CHOOSE THIS?

A case small enough to check every number.

WHY NOT THIS? (TRADEOFFS)

Two points cannot show what happens when the classes overlap.

2. Correctness checks

DEFINITION — start here, no prior knowledge assumed

The **dual problem** gives each point a multiplier α, between 0 and C, with $\Sigma \alpha_i y_i = 0$. **SMO** (sequential minimal optimisation) improves two multipliers at a time, choosing the pair that most violates the optimality conditions, and solves for that pair exactly. The **KKT conditions** say what optimal looks like: $\alpha = 0$ outside the margin, $0 < \alpha < C$ exactly on it, $\alpha = C$ inside it.

REAL-WORLD SCENARIO

Tightening the bolts of a wheel two at a time, always the pair that is furthest off, until none is.

MATHEMATICS

$$\textit{maximise } \Sigma \alpha_i - \tfrac{1}{2} \Sigma_{i,j} \alpha_i \alpha_j y_i y_j \, x_i \cdot x_j, \quad 0 \le \alpha_i \le C, \quad \Sigma \alpha_i y_i = 0$$

$$w = \Sigma \alpha_i y_i x_i; \quad \textit{at the optimum, primal = dual}$$

IMPLEMENTATION

```
def smo(K, y, C, tol=TOL, max_iter=200000):
    n = len(y); a = np.zeros(n); f = np.zeros(n)
    for it in range(max_iter):
        G = y * f - 1
        up = ((a < C) & (y > 0)) | ((a > 0) & (y < 0))
        low = ((a < C) & (y < 0)) | ((a > 0) & (y > 0))
        v = -y * G
        i = np.where(up)[0][np.argmax(v[up])]; j = np.where(low)[0][np.argmin(v[low])]
        if v[i] - v[j] < tol:
            break
        eta = max(K[i, i] + K[j, j] - 2 * K[i, j], 1e-12)
        if y[i] != y[j]:
            L, H = max(0.0, a[j] - a[i]), min(C, C + a[j] - a[i])
        else:
            L, H = max(0.0, a[i] + a[j] - C), min(C, a[i] + a[j])
        Ei, Ej = f[i] - y[i], f[j] - y[j]
        aj = np.clip(a[j] + y[j] * (Ei - Ej) / eta, L, H)
        ai = a[i] + y[i] * y[j] * (a[j] - aj)
        ai, aj = [0.0 if v < 1e-12 * C else C if v > C * (1 - 1e-12) else v for v in (ai, aj)]
        f +=(ai - a[i]) * y[i] * K[:, i] + (aj - a[j]) * y[j] * K[:, j]
        a[i], a[j] = ai, aj
    return a, it + 1
```

ACTUAL OUTPUT

```
SMO finished in 5540 pair updates; sum of a_i y_i = 0.0e+00 (must be 0); every multiplier in [0, C]: True
KKT: points with a = 0 should have margin >= 1: worst 1.0950; points with 0 < a < C should have margin 1:
    range 0.9995 to 1.0005; points with a = C should have margin <= 1: worst 0.9941
primal 19.0334, dual 19.0309: the duality gap is 0.0026, 0.013% of the primal (0 at the exact optimum)
scikit-learn's SVC on the same data: cosine between the two w 0.999998; b -0.2730 against -0.2729; support
    vectors 35 against 35
test predictions that agree with scikit-learn: 1.0000
Actual output from this lesson's run.
```

What it means

Every check passes to within the solver's tolerance of 0.001: margins of 0.9995 to 1.0005 for points on the margin, a duality gap of 0.013%, and the same 35 support vectors as scikit-learn. One honest detail from building this: the first version never finished, because rounding left some multipliers at about 10^{-17} instead of exactly 0, and the solver kept choosing a pair it could not move. The line that snaps values onto their bounds fixed it.

WHY CHOOSE THIS?

SMO needs no general optimisation library, and each step is solved exactly.

WHY NOT THIS? (TRADEOFFS)

It needs the n × n matrix of dot products, so memory grows with the square of the rows.

3. Separable data: the widest gap

DEFINITION — start here, no prior knowledge assumed

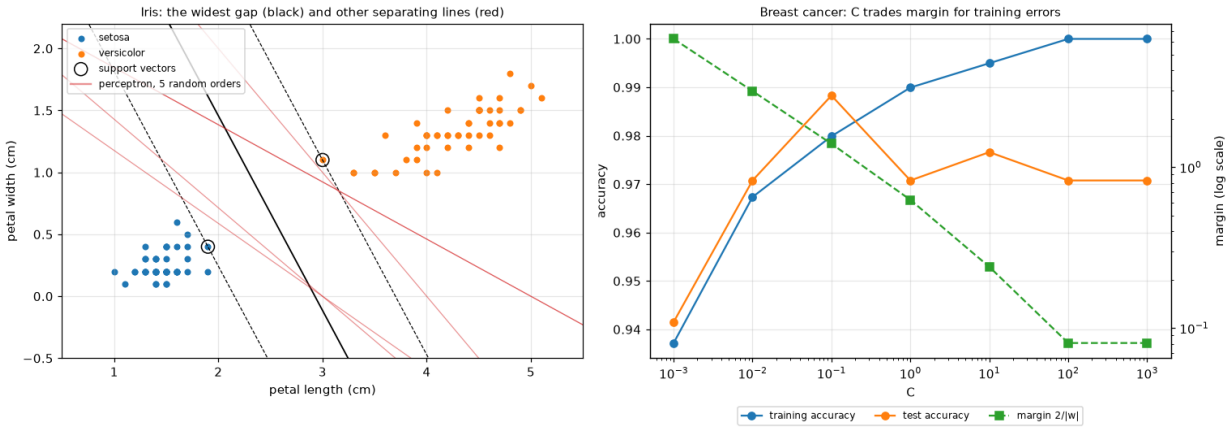
Setosa and versicolor irises are separable by petal size. Many lines separate them; the SVM chooses the one with the widest margin. The **perceptron**, an older method, stops at the first line that separates the training points.

REAL-WORLD SCENARIO

Many routes avoid the potholes; the best one keeps the most distance from all of them.

ACTUAL OUTPUT

```
100 flowers; SVM with C = 1e6 (a hard margin): 2 support vectors, margin width 1.3038 cm, training
    accuracy 1.0000
perceptron from 20 random orders: training accuracy 1.0000 every time: True
  but its gap ranges from 0.0109 to 1.2132 cm (mean 0.5283); the SVM's is the largest possible,
      1.3038
Actual output from this lesson's run.
```



Left: iris, the SVM's boundary and margin (black), and the perceptron from five random orders (red). Right: breast cancer, training and test accuracy and the margin, against C.

What it means

Both methods separate the training flowers perfectly, so training accuracy cannot tell them apart. The margin can: the perceptron's ranges from 0.0109 to 1.2132 cm depending only on the order it saw the flowers, and the SVM's is 1.3038, decided by just 2 support vectors. A boundary that nearly touches a training flower is a guess about the next flower; the widest one is the most cautious guess.

WHY CHOOSE THIS?

A unique, reproducible answer with the most room for error.

WHY NOT THIS? (TRADEOFFS)

On truly separable data a hard margin can be decided by one or two unusual points (section 7).

4. Only the support vectors matter

DEFINITION — start here, no prior knowledge assumed

The weights are $w = \Sigma \alpha_i y_i x_i$, and every point outside the margin has $\alpha = 0$. So retraining without those points should change nothing, while removing support vectors should change the answer.

REAL-WORLD SCENARIO

Removing a fence post far from the road changes nothing; removing one at the edge lets the road shift.

ACTUAL OUTPUT

```
breast cancer, standardised; SMO stops at a tolerance of 0.001, so changes of about that size are the
    solver's, not the data's
C = 0.01: 96 support vectors of 398. Relative change in w: retrained on the support vectors alone 2.3e-04;
    without half of the other points 1.2e-03; without half of the support vectors 2.4e-01
C = 1.0: 35 support vectors of 398. Relative change in w: retrained on the support vectors alone 1.6e-04;
    without half of the other points 1.6e-04; without half of the support vectors 4.9e-01
Actual output from this lesson's run.
```

What it means

The contrast is the point. Retrained on its 35 support vectors alone, out of 398 tumours, w moves by 1.6e-04, about the solver's tolerance. Without half of the other points, also 1.6e-04. Without half of the support vectors, it moves by 4.9e-01: half its size. An SVM is a summary of a few hard cases.

WHY CHOOSE THIS?

A compact model: prediction needs only the support vectors.

WHY NOT THIS? (TRADEOFFS)

Those few points carry all the weight, so their labels had better be right.

5. Non-separable data: what C does

DEFINITION — start here, no prior knowledge assumed

Breast cancer cannot be separated cleanly at a sensible margin. C sets the price of each violation: small C allows many points inside the margin for a wide gap; large C fits the training points tightly.

REAL-WORLD SCENARIO

A strict teacher marks every slip and the students learn to memorise; a lenient one marks only big mistakes.

MATHEMATICS

$$\text{minimise } \tfrac{1}{2}|w|^2 + C \sum \max(0, 1 - y_i(w \cdot x_i + b)) \quad \text{(the hinge loss)}$$

ACTUAL OUTPUT

```
       C   margin  support  inside or wrong  train acc  test acc  SMO steps
   0.001   6.2727      200               192     0.9372    0.9415        113
    0.01   2.9724       96                87     0.9673    0.9708        165
     0.1   1.3990       51                37     0.9799    0.9883        416
       1   0.6268       35                15     0.9899    0.9708       5540
      10   0.2403       29                 6     0.9950    0.9766      23154
     100   0.0808       23                 0     1.0000    0.9708      50870
    1000   0.0808       23                 0     1.0000    0.9708      50870
margin is 2/|w| in standardised units; 'inside or wrong' counts training points with y f(x) < 0.99 (points on
    the margin sit within the solver's tolerance of 1)
Actual output from this lesson's run.
```

What it means

Raising C narrows the margin from 6.2727 to 0.0808 and lifts training accuracy to 1.0000, but the best test score on this split is at C = 0.1 (0.9883). At C ≥ 100 nothing changes: the training points have become separable in 30 dimensions, so the soft margin has turned into a hard one. The SMO steps grow from 113 to 50870 as C rises: a harder problem for the solver too.

WHY CHOOSE THIS?

One knob between underfitting and overfitting.

WHY NOT THIS? (TRADEOFFS)

C has to be chosen by cross-validation, and its best value depends on the scaling.

6. Choosing C, scaling, and logistic regression

DEFINITION — start here, no prior knowledge assumed

Over 20 splits, C is chosen by 5-fold cross-validation inside each training part. The same C is also tried on the raw measurements, and logistic regression gets the same penalty strength.

REAL-WORLD SCENARIO

Two judges with different scoring rules, ranking the same contestants.

ACTUAL OUTPUT

```
C chosen by 5-fold CV on each training part: [(0.01, 1), (0.1, 16), (1, 3)]
linear SVM, standardised: 0.9746 (std 0.0116)
linear SVM, raw measurements, same C: 0.6696 (std 0.2557); SMO hit its 200,000-step limit on 19 of 20 splits
logistic regression, standardised, same penalty: 0.9769 (std 0.0110)
  SVM minus raw       +0.3050 [+0.1974, +0.4161]; better on 20, worse on 0 of 20
  SVM minus logistic  -0.0023 [-0.0070, +0.0020]; better on 5, worse on 11 of 20
the splits share rows, so these intervals are too narrow, a rough guide only
Actual output from this lesson's run.
```

What it means

Two lessons. Scaling is not optional: on raw measurements, where areas run into the thousands and other features are below 1, the SVM scores 0.6696 against 0.9746, and SMO did not even finish on 19 of 20 splits. And the SVM's famous margin buys nothing here over logistic regression (-0.0023, inside the noise): with a linear boundary, the two losses usually reach very similar lines. What the SVM adds is sparsity, and on Day 113, kernels.

WHY CHOOSE THIS?

Strong, well-understood linear classifier with a convex problem.

WHY NOT THIS? (TRADEOFFS)

No probabilities out of the box, and no better than logistic regression on this data.

7. One mislabelled point

DEFINITION — start here, no prior knowledge assumed

One versicolor, the one with the shortest petal, is relabelled as setosa. The data stays separable, so a hard margin must squeeze past it.

REAL-WORLD SCENARIO

One wrong address in a delivery list: a driver who must visit every address takes a strange route; one who may skip a stop does not.

ACTUAL OUTPUT

```
flower 98 (petal 3.0 x 1.1 cm, really versicolor) relabelled as setosa
  C = 1000000.0: margin 0.3162 cm, 2 support vectors, the other 99 flowers right: 1.0000
  C = 10     : margin 0.6325 cm, 2 support vectors, the other 99 flowers right: 1.0000
  C = 1      : margin 1.5040 cm, 4 support vectors, the other 99 flowers right: 1.0000
  C = 0.1    : margin 2.1189 cm, 12 support vectors, the other 99 flowers right: 1.0000
Actual output from this lesson's run.
```

What it means

The hard margin shrinks from 1.3038 cm to 0.3162 cm to accommodate a single error; with C = 1 it is back to 1.5040, wider than before, because the soft margin can treat the mislabelled flower as a violation instead. All other flowers stay correct in every case here, so this shows the mechanism, not a loss of accuracy.

WHY CHOOSE THIS?

The soft margin limits the damage any one point can do.

WHY NOT THIS? (TRADEOFFS)

Too small a C would ignore real structure as if it were noise.

Interview preparation — technical questions

Q: What is a support vector?

A: A training point with a nonzero multiplier: on the margin or inside it. Only these points determine w and b.

Q: How does SMO work?

A: It updates two multipliers at a time (one cannot move alone, since the multipliers must keep sum a y = 0), choosing the most violating pair, until none violates optimality.

Q: What does C control?

A: The price of margin violations: here margin 6.2727 at C 0.001 against 0.0808 at C 100.

Q: Why must you scale features for an SVM?

A: The margin is measured in feature units: unscaled, accuracy fell to 0.6696 and SMO did not converge.

Q: How do you check an SVM solver?

A: KKT conditions, the duality gap, and agreement with a trusted implementation.

Interview preparation — theoretical questions

Q: Why maximise the margin?

A: Among all separating lines it is the most robust to small changes in new points; the perceptron's gap varied from 0.0109 to 1.2132.

Q: Why solve the dual?

A: It depends on the data only through dot products, which is what makes the kernel trick possible (Day 113).

Q: SVM or logistic regression?

A: With a linear boundary they are often equivalent (-0.0023 here); SVMs are sparse, logistic regression gives probabilities.

Q: What is the hinge loss?

A: max(0, 1 - y f(x)): zero for points safely outside the margin, linear inside it.

Key takeaway and what's next

KEY TAKEAWAY

An SVM is the widest-margin boundary, and the dual problem makes it a function of a handful of support vectors. From scratch it matched scikit-learn on every test tumour. Its real strengths are a unique answer and sparsity; its accuracy on this linear problem was no better than logistic regression, and it needs scaled features and a well-chosen C.

- Correctness: duality gap 0.013%; 35 support vectors, as in scikit-learn; test agreement 1.0000.
- Iris: margin 1.3038 cm from 2 support vectors; perceptron 0.0109 to 1.2132 cm.
- Without half the support vectors w changes by 4.9e-01; without other points, by about the solver's tolerance.
- Breast cancer: SVM 0.9746, logistic regression 0.9769, unscaled 0.6696.

Suggested watch order, if starting from zero

- 1. Support Vector Machines Part 1 (of 3): Main Ideas!!! (StatQuest) — the geometry: margins, support vectors and soft margins.
- 2. Support Vector Machines in Python from Start to Finish (StatQuest) — the same ideas in scikit-learn on a real dataset.

Day 113 preview

- **The kernel trick.** The dual only uses dot products: replace them with a kernel, and the same solver draws curved boundaries.